RED & WHITE SKILL EDUCATION

DEEP LEARNING

PR 3

Project Definition

Topics: Morphological Operations · Bitwise Operations · Histograms · Face Detection (YuNet) · YOLO Object Detection

Total Marks: 10

Field	Detail
Institute	Red & White Skill Education
Subject	Deep Learning
Project	PR 3
Total Marks	10
Topics	Morphological Operations (Dilation, Erosion, Opening, Closing) · Bitwise Operations (AND, OR, XOR, NOT) · Image Histograms (Gray & Color, Brightness & Contrast) · Face Detection (YuNet) · Object Detection with YOLO
Dataset	Custom OpenCV sample image pack + pretrained YuNet face detector + pretrained YOLOv8 (COCO) weights
Task Type	Classical Image Processing → Face Detection → Deep-Learning-based Object Detection (integrated real-time CV pipeline)
Framework	OpenCV (cv2) 5.0.0 + Ultralytics YOLOv8 + NumPy + Matplotlib (Python 3.x, Jupyter Notebook)
Project Duration	3 Days (2 Hours per Day)

§ 1 · Dataset

This project moves from classical, hand-engineered image-processing operations to modern deep-learning-based detection, all within OpenCV's ecosystem. Rather than a single labelled dataset, you will work across three complementary data sources: a small pack of static test images for pixel-level operations (morphology, bitwise logic, histograms), OpenCV's bundled pretrained face detectors, and Ultralytics' pretrained YOLOv8 weights trained on the 80-class COCO dataset for object detection. This mirrors how real CV pipelines are built in industry — surveillance, retail analytics, and quality inspection systems typically combine cheap classical pre-processing with a deep detector rather than relying on either alone.

Field	Detail
Static Image Pack	6–10 sample images: one noisy/textured image for morphology, one image with 2–3 solid-colour objects on a plain background for bitwise masking, and 3–4 general photos containing people and everyday objects (person, car, bottle, chair, etc.) for histograms and detection
Recommended Source	OpenCV's own sample data repository — https://github.com/opencv/opencv/tree/master/samples/data (e.g. lena.jpg, baboon.jpg, coins.png) — supplemented with 2–3 of your own phone photos
Face Detector Weights	face_detection_yunet_2023mar.onnx — OpenCV 5.0's bundled YuNet detector, loaded via cv2.FaceDetectorYN_create(), downloadable from the OpenCV Zoo (https://github.com/opencv/opencv_zoo)

Field	Detail
Object Detector Weights	yolov8n.pt — Ultralytics YOLOv8-Nano, pretrained on COCO (80 classes) — auto-downloads on first use via `from ultralytics import YOLO; YOLO('yolov8n.pt')`
Live Input	Laptop webcam (cv2.VideoCapture(0)) for the real-time face-detection, YOLO-detection, and integrated-pipeline tasks — a pre-recorded video may be substituted if a webcam is unavailable
License	OpenCV sample data: BSD-3-Clause · Ultralytics YOLOv8: AGPL-3.0 (pretrained weights free for academic/educational use)

Note: Install dependencies once at the start: pip install opencv-python==5.0.0.* opencv-contrib-python==5.0.0.* ultralytics matplotlib numpy. Keep all static images in a /data/images folder and downloaded model weights (including face_detection_yunet_2023mar.onnx) in /data/models, so paths stay consistent across the notebook.

Business Context: This pipeline mirrors a low-cost smart-surveillance or retail-analytics system — classical morphology and bitwise pre-processing clean up noisy camera frames cheaply, and the two deep-learning detectors (face + YOLO) provide accurate identification on top — a pattern widely used in edge-deployed CV systems in India's security and smart-retail sectors.

§ 2 · Project Tasks — What to Do

Complete ALL tasks in a single Jupyter Notebook named CV_PR3.ipynb. Structure the notebook with Markdown headings matching the task titles below. Each task builds on the previous — do not skip or reorder. Your goal is to understand WHY each technique is chosen at each stage of the pipeline, not just how to call the OpenCV / YOLO function.

Task 1: Environment Setup, Image I/O & Morphological Operations

#	Action	Details / Configuration
1.1	**Environment setup & image loading**	pip install opencv-python==5.0.0.* opencv-contrib-python==5.0.0.* ultralytics matplotlib numpy. import cv2, numpy as np, matplotlib.pyplot as plt. Print cv2.__version__ to confirm OpenCV 5.0.0 is active. Load one grayscale and one colour image with cv2.imread(); print img.shape and img.dtype. Write a helper show(img, title) using matplotlib (convert BGR→RGB for colour images) since cv2.imshow does not render inline in Jupyter.
1.2	**Binarize a test image**	Convert a suitable image to grayscale (cv2.cvtColor(..., COLOR_BGR2GRAY)) and threshold it to binary with cv2.threshold(img, 127, 255, cv2.THRESH_BINARY) to create the input for morphological experiments. Display original vs binary side-by-side.
1.3	**Apply the four morphological operations**	Define a 5×5 rectangular kernel: kernel = np.ones((5,5), np.uint8). Apply cv2.erode(), cv2.dilate(), and cv2.morphologyEx() with MORPH_OPEN and MORPH_CLOSE. Plot all four results plus the original in a 1×5 grid with titles.
1.4	**Kernel shape & size comparison**	Repeat opening/closing using cv2.getStructuringElement() with MORPH_RECT, MORPH_ELLIPSE, and MORPH_CROSS at 3×3, 5×5, and 9×9 sizes on a noisy binary image. Plot a 3×3 grid (kernel shape × size) and note which combination best removes salt-and-pepper noise without eroding the object outline.
1.5	**WHY morphology works — Markdown**	Add a Markdown cell: erosion is a local minimum filter (shrinks white regions, removes small bright noise); dilation is a local maximum filter (grows white regions, fills small dark gaps). Opening = erosion → dilation, removing small noise while preserving overall shape/size. Closing =

#	Action	Details / Configuration
		dilation → erosion, filling small holes without shrinking the object. State a real scenario where you would use each before passing a mask to a downstream detector.

Task 2: Bitwise Operations & Image Histograms

#	Action	Details / Configuration
2.1	**Bitwise AND / OR / XOR / NOT**	Create two binary masks with cv2.circle() and cv2.rectangle() on blank np.zeros canvases. Apply cv2.bitwise_and(), cv2.bitwise_or(), cv2.bitwise_xor(), and cv2.bitwise_not() between them. Plot a 2×3 grid: the two source masks plus the four results.
2.2	**Applied use-case: masked overlay**	Use one of the masks (e.g. a circular ROI) with bitwise_and against a real photo to extract only the masked region, and bitwise_or to composite a logo/watermark onto a background image. Display before/after.
2.3	**Grayscale & per-channel colour histograms**	Compute cv2.calcHist([gray],[0],None,[256],[0,256]) and plot with matplotlib. For a colour image, compute and overlay B, G, R histograms on one axes (blue/green/red lines) using cv2.calcHist per channel.
2.4	**Brightness & contrast adjustment**	Apply cv2.convertScaleAbs(img, alpha=1.5, beta=30) (higher contrast + brighter) and alpha=0.7, beta=-30 (lower contrast + darker). Display the three versions (original, brightened, darkened) with their grayscale histograms stacked below each image.
2.5	**WHY alpha/beta and histograms matter — Markdown**	Explain: new_pixel = alpha·pixel + beta — alpha (contrast gain) stretches or compresses the histogram spread around its mean, while beta (brightness offset) shifts the whole histogram left/right. A narrow histogram = low contrast (washed out); a histogram clipped at 0 or 255 = over/under-exposure. Note how this diagnostic view helps you judge whether an image needs pre-processing before being fed to the face/object detectors in later tasks.

Task 3: Face Detection with YuNet (OpenCV 5.0)

#	Action	Details / Configuration
3.1	**Load the YuNet face detector**	Download face_detection_yunet_2023mar.onnx from the OpenCV Zoo. Create the detector with detector = cv2.FaceDetectorYN_create('face_detection_yunet_2023mar.onnx', '', (img_w, img_h), score_threshold=0.9). Call detector.setInputSize((img_w, img_h)) to match each image's dimensions, then _, faces = detector.detect(img).
3.2	**Detect on static images**	Run the detector on 3–4 static images with varied pose, lighting, and number of people (include at least one side-profile face and one with partial occlusion / sunglasses). Each row of faces gives box (x, y, w, h), 5 landmark points, and a confidence score — draw the box, landmarks, and score with cv2.rectangle()/cv2.circle().
3.3	**Real-time detection on webcam**	Loop cv2.VideoCapture(0), calling detector.setInputSize() once per frame size and detector.detect(frame) on every frame, drawing boxes/landmarks/confidence live. Report the achieved FPS.
3.4	**Score-threshold experiment & applied privacy use-case**	Re-run detection at score_threshold = 0.5, 0.7, 0.9 and note how the number of detected faces and false positives changes. Then, using the detected boxes, apply cv2.GaussianBlur() only within each face region (a practical anonymisation / privacy-blur filter) and display before/after.
3.5	**WHY YuNet — Markdown**	Explain that YuNet is a compact, modern CNN-based detector (the default recommended face detector shipped with OpenCV 5.0 via

#	Action	Details / Configuration
		cv2.FaceDetectorYN) trained on diverse, in-the-wild faces, giving it strong generalisation to profile views, partial occlusion, and small/blurry faces, while also returning 5-point facial landmarks in a single forward pass. Discuss how the score_threshold trades off missed detections against false positives, and what threshold you would pick for a real-time security camera versus an offline photo-tagging tool.

Task 4: Object Detection with YOLO

#	Action	Details / Configuration
4.1	**Load pretrained YOLOv8 & run on static images**	from ultralytics import YOLO; model = YOLO('yolov8n.pt'). Run model(image_path) on 2–3 photos containing multiple COCO-class objects (person, car, dog, chair, etc.). Use results[0].plot() to render boxes, class labels, and confidence scores; display with matplotlib.
4.2	**Real-time YOLO on webcam**	Loop cv2.VideoCapture(0), passing each frame to model(frame, verbose=False), rendering results[0].plot(), and displaying/saving the annotated stream. Record and report the achieved FPS.
4.3	**Confidence & IoU threshold tuning**	Re-run inference at conf=0.25, 0.5, 0.75 and iou=0.3, 0.5, 0.7 (model(img, conf=..., iou=...)). Plot how the number of retained boxes changes with each threshold, and show one example where a low IoU setting causes duplicate overlapping boxes for the same object (before Non-Max Suppression cleans it up at a higher threshold).
4.4	**Per-class detection summary**	Across all test images/frames, tally how many detections fall into each COCO class detected (e.g. person: 5, car: 2, dog: 1) and plot as a bar chart. Identify any classes that were missed or misclassified and discuss possible reasons (occlusion, small size, unusual angle).
4.5	**WHY YOLO generalises — Markdown**	Explain YOLO's single-stage, grid-based design: the image is divided into a grid, and each cell directly regresses bounding boxes + class probabilities in one forward pass, followed by Non-Max Suppression to remove duplicate detections. Discuss the accuracy-vs-speed trade-off of the 'nano' variant used here versus larger YOLOv8 variants (s/m/l/x), and why a single pretrained model can detect 80 different object classes without any per-class hand-tuning.

Task 5: Integrated Pipeline & Final Comparison

#	Action	Details / Configuration
5.1	**Build the unified pipeline**	On a single webcam stream, run per frame: (a) YuNet face detection (Task 3) in green boxes, (b) YOLO object detection (Task 4) in red boxes with class labels. Overlay both annotation layers on the same output frame.
5.2	**Morphological pre-cleaning**	Before running detection, optionally apply a light morphological opening (Task 1) to denoise the raw camera frame, and compare detection quality/count with and without this pre-processing step on a noisy frame.
5.3	**FPS benchmarking**	Measure and record average FPS for: face detection alone, YOLO alone, and the full combined pipeline, each over the same 100-frame sample. Plot a bar chart of the three FPS values.
5.4	**Final results comparison table**	Build a table: Technique | Type (Classical / Deep Learning) | What It Detects | Lighting Robustness | Approx. FPS | Best Use Case. Include every technique from Tasks 1–4 (morphology, bitwise/histograms, YuNet face, YOLO).

#	Action	Details / Configuration
5.5	**Reflection — Markdown**	Add a closing Markdown cell: which combination of classical pre-processing + deep-learning detection gave the best accuracy-to-compute trade-off for a real-time monitoring use case, and what would you change (e.g. swap yolov8n for yolov8s, or adjust the YuNet score_threshold) if deploying on a low-power edge device versus a cloud server.

§ 3 · Transparent Marking Scheme (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb)	**5 Marks**	50%	Completed notebook — all 5 tasks, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	**3 Marks**	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	**2 Marks**	20%	Public repo with .ipynb + .html + plots/clips + README + video link
TOTAL	**10 Marks**	**100%**	

NOTE:

- Each late submission will decrease the marks by 1 per one day.
- For example, if you are 2 days late submitting the project, your 2 marks will be cut.
- So respect the given deadline, and according to that, submit the project within the given timeline.

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	**1.0**	Morphology fully explored: 4 ops on binary image, kernel shape/size 3×3 grid, Markdown on erosion/dilation as min/max filters	4 ops shown; 1–2 kernel variations; brief Markdown	2–3 ops shown; minimal kernel comparison	1 op only; no comparison	Not done
A2	**1.0**	Bitwise ops (AND/OR/XOR/NOT) with masked overlay use-case; gray + colour histograms; brightness/contrast with histogram comparison and alpha/beta Markdown	Bitwise + histograms done; contrast adjustment done; minor Markdown gaps	Bitwise or histograms only, not both fully	Only bitwise OR only histograms attempted	Not done
A3	**1.0**	YuNet face detector run on static images and webcam with landmarks + confidence; score-threshold experiment (0.5/0.7/0.9); blur/anonymise use-case; YuNet mechanism Markdown	Detector run on images and webcam; threshold experiment or use-case incomplete	Detector run on static images only	Attempted, non-functional	Not done
A4	**1.5**	YOLOv8 run on images + webcam with FPS reported; confidence/IoU threshold experiment with duplicate-box	YOLO run on images and webcam; threshold	YOLO run on static images only	Attempted, non-functional / wrong classes	Not done

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
		example; per-class detection summary; single-stage-detector Markdown	experiment or class summary incomplete			
A5	**0.5**	Both annotation layers (face + YOLO) combined on one stream; FPS bar chart for all 3 configurations; full comparison table across Tasks 1–4; reflection Markdown	Pipeline combines both layers; FPS or comparison table partial	Only 1 layer running in the 'integrated' pipeline	Attempted, non-functional	Not done

Subtotal A: 1.0 + 1.0 + 1.0 + 1.5 + 0.5 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your Jupyter Notebook / live-demo screen simultaneously. Explain concepts verbally — do not just run cells without speaking.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	**1.0**	Clear audio and video throughout; face visible via PiP; notebook outputs, live webcam demos, and all plots clearly visible; all 5 tasks demonstrated in order	Clear audio; face mostly visible throughout	Audio unclear; face partially visible	Very poor quality, hard to follow	No video
B2	**1.5**	Verbally explains all 4 core concepts with reasoning: (1) erosion/dilation as min/max filters and why opening vs closing is chosen; (2) alpha/beta effect on histograms and brightness/contrast; (3) YuNet's CNN-based detection mechanism, landmarks, and the score-threshold trade-off; (4) YOLO's single-stage grid + NMS design and the accuracy/speed trade-off of model size	Explains 3 concepts with reasoning	Demonstrates steps without explaining theory	Reads code from screen; no conceptual depth	No verbal explanatio n
B3	**0.5**	5–10 min duration; covers all 5 tasks; shows the live webcam face-detection and YOLO demos running; shows final FPS chart and results table	4–5 min; covers most tasks	2–4 min; misses key demos	Under 2 min or very incomplete	Not submitted

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Video Requirements

Field	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Jupyter Notebook — show morphology/histogram plots, live face-detection webcam demo, live YOLO webcam demo, final comparison table, FPS bar chart
Duration	5 to 10 minutes
Naming convention	CV_PR3_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (2 Marks)

Create a public GitHub repository specifically for this project.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	**0.5**	Repo contains: CV_PR3.ipynb + CV_PR3.html + /plots folder with all key figures clearly named (morphology_grid.png, histogram_comparison.png, face_detection_comparison.png, yolo_detections.png, fps_comparison.png, results_table.png) + requirements.txt	Key files present; a few plots missing or unnamed	Only .ipynb uploaded; no /plots folder	Repo exists but empty or broken link	No GitHub repo
C2	**1.0**	README contains: project title + all data/model sources with links + preprocessing/setup steps + all 5 topics with 2-line descriptions + /plots screenshots with captions + full comparison table + video link + tools section	Title + sources + partial topic docs; some plots or results missing	Title and repo URL only; no topic descriptions or results	Blank README or single line	No README
C3	**0.5**	3+ meaningful commits, e.g.: 'Add morphology, bitwise & histogram tasks'; 'Add YuNet face detection task'; 'Add YOLO detection & integrated pipeline with FPS benchmarking'	2–3 adequate commits with basic messages	1 bulk commit with all files at once	Commit messages unclear or auto-generated	No commits or single initial commit

Subtotal C: 0.5 + 1.0 + 0.5 = 2.0 Marks

§ 4 · Submission Checklist

Tick every item below before submitting. All items are required for full marks.

- opencv-python 5.0.0, opencv-contrib-python 5.0.0, ultralytics, matplotlib, numpy installed; cv2.__version__ confirmed as 5.0.0
- Sample images loaded from /data/images; grayscale + colour images confirmed with .shape and .dtype
- All 4 morphological operations (erode, dilate, open, close) applied and plotted in a 1×5 grid
- Kernel shape (rect/ellipse/cross) × size (3/5/9) 3×3 comparison grid completed
- Markdown: erosion/dilation as min/max filters; opening vs closing use-case explained
- Bitwise AND/OR/XOR/NOT demonstrated on two masks; masked overlay use-case shown
- Grayscale histogram and overlaid B/G/R colour histograms plotted
- Brightness/contrast adjustment (convertScaleAbs) with before/after histograms; alpha/beta Markdown
- YuNet face detector (cv2.FaceDetectorYN, OpenCV 5.0) run on static images with confidence scores and landmarks shown
- YuNet run in real time on webcam; FPS measured and reported
- Score-threshold experiment (0.5/0.7/0.9) completed; effect on detections and false positives noted
- Face-region blur/anonymisation use-case demonstrated
- Markdown: YuNet's CNN-based detection mechanism and score-threshold trade-off explained
- YOLOv8n loaded and run on static images with rendered boxes/labels/confidence
- YOLOv8n run in real time on webcam; FPS measured and reported
- Confidence and IoU threshold experiment completed; duplicate-box / NMS example shown
- Per-class detection summary bar chart completed
- Markdown: YOLO's single-stage grid + NMS design and model-size trade-off explained
- Integrated pipeline combines face detection and YOLO annotation layers on one webcam stream

- FPS benchmarked separately (face, YOLO) and combined; bar chart plotted
- Full comparison table built across all techniques from Tasks 1–4
- Closing reflection Markdown on best classical + deep-learning combination for edge vs cloud deployment
- Every task has an opening Markdown cell: title + objective (1 sentence) + expected output
- All plots: descriptive title, labelled axes, legend where relevant, consistent styling
- Kernel restarted — Restart & Run All executed with zero errors before submission
- Notebook exported as HTML — both CV_PR3.ipynb and CV_PR3.html present in repository
- requirements.txt: opencv-python==5.0.0.*, opencv-contrib-python==5.0.0.*, ultralytics, numpy, matplotlib
- Video: face (PiP) + screen, 5–10 min, MP4/MOV, named CV_PR3_YourName_GRID.mp4
- Video explains all 4 concepts verbally, including live webcam face-detection and YOLO demos
- GitHub: public repository with CV_PR3.ipynb + CV_PR3.html + /plots folder + requirements.txt
- README.md: data/model sources + 5 topic descriptions + plots with captions + results table + video link
- 3+ meaningful Git commits with descriptive messages referencing specific tasks
- Video link pasted in GitHub README.md under the Video section

© 2026 Red & White Skill Education. All Rights Reserved.